



R.M.K. College of Engineering and Technology

(An Autonomous Institution)

RSM Nagar, Puduvoyal- 601206, Gummidipoondi (T.K), Thiruvallur (D.T), Tamil Nadu

Approved by AICTE, New Delhi/ Affiliated to Anna University, Chennai

Accredited by NBA (All Eligible Courses) / NAAC with "A" GRADE

An ISO 21001:2018 Certified Institution

Department of Artificial Intelligence and Data Science



VISION AND MISSION OF THE INSTITUTE

VISION

To be a globally recognized technological Institution fostering excellence in academics, research, and entrepreneurship, producing socially responsible engineers to meet evolving technological changes leading to the sustainable development of the nation and the world.

MISSION

- ❖ To impart quality technical education through outcome-based learning and modern teaching practices.
- ❖ To promote research, innovation, and entrepreneurship through strong industry interaction and incubation initiatives.
- ❖ To enhance employability and promote sustainability through continuous skill development and responsible social engagement.

VISION AND MISSION OF THE DEPARTMENT

VISION

To nurture a globally competitive Artificial Intelligence community, by delivering quality education, fostering research through innovative technologies and in-turn contributing to the sustainable growth of the nation.

MISSION

- M1:** To provide quality engineering education, implementing outcome-based education and innovative teaching-learning practices
- M2:** To strengthen research through innovation and industrial collaboration.
- M3:** To foster engineering professionals with a commitment to lifelong learning and sustainable development.



R.M.K. College of Engineering and Technology

(An Autonomous Institution)

RSM Nagar, Puduvoyal- 601206, Gummidipoondi (T.K), Thiruvallur (D.T), Tamil Nadu

Approved by AICTE, New Delhi/ Affiliated to Anna University, Chennai

Accredited by NBA (All Eligible Courses) / NAAC with "A" GRADE

An ISO 21001:2018 Certified Institution

Department of Artificial Intelligence and Data Science



PROGRAM EDUCATIONAL OBJECTIVES (PEOS)

Graduates of Artificial Intelligence & Data Science will

PEO 1: Design, develop, and deploy AI and data-driven solutions using modern tools and technologies

PEO 2: Demonstrate professional competence through research, innovation, adapting emerging technologies

PEO 3: Exhibit leadership and professionalism in their career

PEO 4: Develop ethical and socially responsible AI solutions contributing to societal welfare and sustainability

PROGRAM SPECIFIC OUTCOMES (PSOS)

Graduates of Artificial Intelligence and Data Science Program will be able to:

PSO 1: Apply Machine Learning and Deep Learning algorithms to design intelligent systems that solve real-world problems.

PSO 2: Employ data analytics, visualization, and predictive modelling tools to support data-driven decision-making.

PSO 3: Design and implement Natural Language Processing models for tasks such as text classification, sentiment analysis, and information extraction.



R.M.K. College of Engineering and Technology

(An Autonomous Institution)

RSM Nagar, Puduvoyal- 601206, Gummidipoondi (T.K), Thiruvallur (D.T), Tamil Nadu

Approved by AICTE, New Delhi/ Affiliated to Anna University, Chennai

Accredited by NBA (All Eligible Courses) / NAAC with "A" GRADE

An ISO 21001:2018 Certified Institution

Department of Artificial Intelligence and Data Science



PROGRAMME OUTCOMES

- 1. Engineering Knowledge:** Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- 2. Problem Analysis:** Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
- 3. Design/ Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.
- 4. Conduct investigations of complex problems** using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.
- 5. Modern Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The Engineer and Society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
- 7. Environment and Sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
- 9. Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.
- 11. Project Management and Finance:** Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long Learning:** Recognize the need for and have the preparation and ability to Engage in independent and life- long learning in the broadest context of technological Change.



R.M.K. College of Engineering and Technology

(An Autonomous Institution)

RSM Nagar, Puduvoyal- 601206, Gummidipoondi (T.K), Thiruvallur (D.T), Tamil Nadu

Approved by AICTE, New Delhi/ Affiliated to Anna University, Chennai

Accredited by NBA (All Eligible Courses) / NAAC with "A" GRADE

An ISO 21001:2018 Certified Institution

Department of Artificial Intelligence and Data Science



22AI704 – LARGE LANGUAGE MODELS (Lab Integrated)

OBJECTIVES:

- Understand text preprocessing, tokenization, and vector representations used in NLP and LLMs.
- Learn the basic concepts of language models used in Large Language Models (LLMs).
- Understand Transformer architecture and Large Language Models for text generation tasks.
- Use pre-trained Hugging Face models for NLP applications and fine-tuning.
- Build LLM-powered applications using Prompt Engineering, RAG and PEFT techniques.

LIST OF EXERCISES:

1. Perform text preprocessing using NLTK/spaCy including tokenization, stop-word removal and lemmatization.
2. Train Word2Vec embeddings using Gensim and visualize semantic similarity between words.
3. Implement an N-gram language model and calculate perplexity for sample text.
4. Train CBOW and Skip-Gram models using Word2Vec and compare semantic similarity results.
5. Implement scaled dot-product attention and visualize attention weights using PyTorch.
6. Build a mini GPT-style Transformer model for text generation.
7. Fine-tune DistilBERT for sentiment classification using HuggingFace Transformers.
8. Build a Named Entity Recognition (NER) model using BERT and HuggingFace Trainer API.
9. Build a Retrieval-Augmented Generation (RAG) pipeline using FAISS/Chroma and Hugging Face embeddings.
10. Fine-tune an open-source LLM using LoRA or QLoRA on a small custom dataset.

OUTCOMES:

Upon completion of the course, the students will be able to:

- CO1:** Describe NLP preprocessing, tokenization, and embedding techniques used in language models.
- CO2:** Apply statistical and neural language modeling techniques as foundations for Large Language Models.
- CO3:** Analyze Transformer architecture and implement Large Language Models for text generation tasks.
- CO4:** Implement and fine-tune pre-trained Hugging Face models for NLP applications.
- CO5:** Develop LLM-powered systems using Prompt Engineering, RAG, and PEFT techniques.
- CO6:** Evaluate ethical issues such as bias, hallucination, and privacy in Large Language Models.



R.M.K. College of Engineering and Technology

(An Autonomous Institution)

RSM Nagar, Puduvoyal- 601206, Gummidipoondi (T.K), Thiruvallur (D.T), Tamil Nadu

Approved by AICTE, New Delhi/ Affiliated to Anna University, Chennai

Accredited by NBA (All Eligible Courses) / NAAC with "A" GRADE

An ISO 21001:2018 Certified Institution

Department of Artificial Intelligence and Data Science



INDEX

Ex.No	DATE	TITLE OF THE PROGRAM	PAGE NO	MARKS	SIGN
1.		Perform text preprocessing using NLTK/spaCy including tokenization, stop-word removal and lemmatization			
2.		Train Word2Vec embeddings using Gensim and visualize semantic similarity between words.			
3.		Implement an N-gram language model and calculate perplexity for sample text.			
4.		Train CBOW and Skip-Gram models using Word2Vec and compare semantic similarity results.			
5.		Implement scaled dot-product attention and visualize attention weights using PyTorch.			
6.		Build a mini GPT-style Transformer model for text generation.			
7.		Fine-tune DistilBERT for sentiment classification using HuggingFace Transformers.			
8.		Build a Named Entity Recognition (NER) model using BERT and HuggingFace Trainer API.			
9.		Build a Retrieval-Augmented Generation (RAG) pipeline using FAISS/Chroma and Hugging Face embeddings.			
10.		Fine-tune an open-source LLM using LoRA or QLoRA on a small custom dataset.			

Ex. No: 1	Text preprocessing warm-up using Python (NLTK / spaCy): Tokenization, Sentence splitting, Lemmatization, Stop-word removal and Word frequency analysis on a small corpus
Date:	

Aim:

To perform basic Natural Language Processing (NLP) text preprocessing techniques such as sentence splitting, tokenization, stop-word removal, lemmatization, and word frequency analysis using NLTK and spaCy.

Algorithm:

1. Start the program.
2. Import the required libraries: NLTK, spaCy, Counter, stopwords, and tokenization functions.
3. Download the required NLTK resources such as punkt and stopwords.
4. Load the spaCy English language model.
5. Define a small text corpus related to Natural Language Processing.
6. Perform sentence splitting using `sent_tokenize()` to divide the text into individual sentences.
7. Perform tokenization using `word_tokenize()` to split the text into individual words and tokens.
8. Remove stop words such as *is, the, it*, etc., and retain only alphabetic words.
9. Perform lemmatization using spaCy to convert words into their base forms.
10. Calculate word frequency using `Counter()` to determine how many times each word occurs.
11. Display the original text, sentences, tokens, filtered words, lemmatized words, and word frequencies.
12. Stop the program.

Program:

```
import nltk
import spacy
from collections import Counter
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize

# Download NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

Load spaCy English model

```
nlp = spacy.load("en_core_web_sm")
```

Small corpus

```
text = """
```

Natural Language Processing (NLP) is a field of Artificial Intelligence.

It helps computers understand human language.

Text preprocessing is an important step in NLP.

```
"""
```

```
print("ORIGINAL TEXT:\n")
```

```
print(text)
```

1. Sentence Splitting

```
sentences = sent_tokenize(text)
```

```
print("\n1. SENTENCE SPLITTING:\n")
```

```
for i, sentence in enumerate(sentences, start=1):
```

```
    print(f'Sentence {i}: {sentence}')
```

2. Tokenization

```
tokens = word_tokenize(text)
```

```
print("\n2. TOKENIZATION:\n")
```

```
print(tokens)
```

3. Stop-word Removal

```
stop_words = set(stopwords.words('english'))
```

```
filtered_tokens = [
```

```
    word for word in tokens
```

```
    if word.lower() not in stop_words and word.isalpha()
```

```
]
```

```
print("\n3. STOP-WORD REMOVAL:\n")
```

```
print(filtered_tokens)
```

4. Lemmatization using spaCy

```
doc = nlp(" ".join(filtered_tokens))
```

```
lemmatized_words = [token.lemma_ for token in doc]
```

```
print("\n4. LEMMATIZATION:\n")
```

```
print(lemmatized_words)
```

5. Word Frequency Analysis

```
word_freq = Counter(lemmatized_words)
print("\n5. WORD FREQUENCY ANALYSIS:\n")
for word, freq in word_freq.items():
    print(f'{word} : {freq}')
```

Output:

```
ORIGINAL TEXT:

Natural Language Processing (NLP) is a field of Artificial Intelligence.
It helps computers understand human language.
Text preprocessing is an important step in NLP.

1. SENTENCE SPLITTING:

Sentence 1:
Natural Language Processing (NLP) is a field of Artificial Intelligence.
Sentence 2: It helps computers understand human language.
Sentence 3: Text preprocessing is an important step in NLP.

2. TOKENIZATION:

['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'helps', 'computers', 'understand', 'human', 'language', '.', 'Text', 'preprocessing', 'is', 'an', 'important', 'step', 'in', 'NLP', '.']

3. STOP-WORD REMOVAL:

['Natural', 'Language', 'Processing', 'NLP', 'field', 'Artificial', 'Intelligence', 'helps', 'computers', 'understand', 'human', 'language', 'Text', 'preprocessing', 'important', 'step', 'NLP']

4. LEMMATIZATION:

['Natural', 'Language', 'Processing', 'NLP', 'field', 'Artificial', 'Intelligence', 'help', 'computer', 'understand', 'human', 'language', 'text', 'preprocesses', 'important', 'step', 'NLP']

5. WORD FREQUENCY ANALYSIS:

Natural : 1
Language : 1
Processing : 1
NLP : 2
field : 1
Artificial : 1
Intelligence : 1
help : 1
computer : 1
understand : 1
human : 1
language : 1
text : 1
preprocesses : 1
important : 1
step : 1
```

Result:

The given text was successfully preprocessed using **NLTK** and **spaCy**. Sentence splitting, tokenization, stop-word removal, lemmatization, and word frequency analysis were performed successfully. The frequency of each lemmatized word was also identified.

Ex. No: 2	Train Word2Vec (Skip-gram) embeddings on a small domain corpus using Genism; visualize nearest neighbours and word analogies using t-SNE / PCA
Date:	

Aim:

To train a Word2Vec Skip-gram model on a small sample corpus and to visualize the learned word embeddings, their nearest neighbours, and word relationships using PCA-based dimensionality reduction.

Algorithm:

1. Start the program.
2. Import the required libraries: Word2Vec from gensim, PCA from sklearn.decomposition, and matplotlib.pyplot.
3. Define a small sample corpus as a list of tokenized sentences.
4. Train a Word2Vec model on the corpus using the Skip-gram algorithm (sg=1) with the specified vector size, window size, and minimum word count.
5. Use the trained model to find and print the words most similar to a chosen target word ("learning").
6. Extract the vocabulary words learned by the model using model.wv.index_to_key.
7. Retrieve the embedding vectors corresponding to the vocabulary words.
8. Apply PCA to reduce the high-dimensional word vectors to two dimensions.
9. Create a scatter plot of the reduced 2D vectors using matplotlib.
10. Annotate each point on the plot with its corresponding word label.
11. Display the plot to visualize word relationships and clustering.
12. Stop the program.

Program:

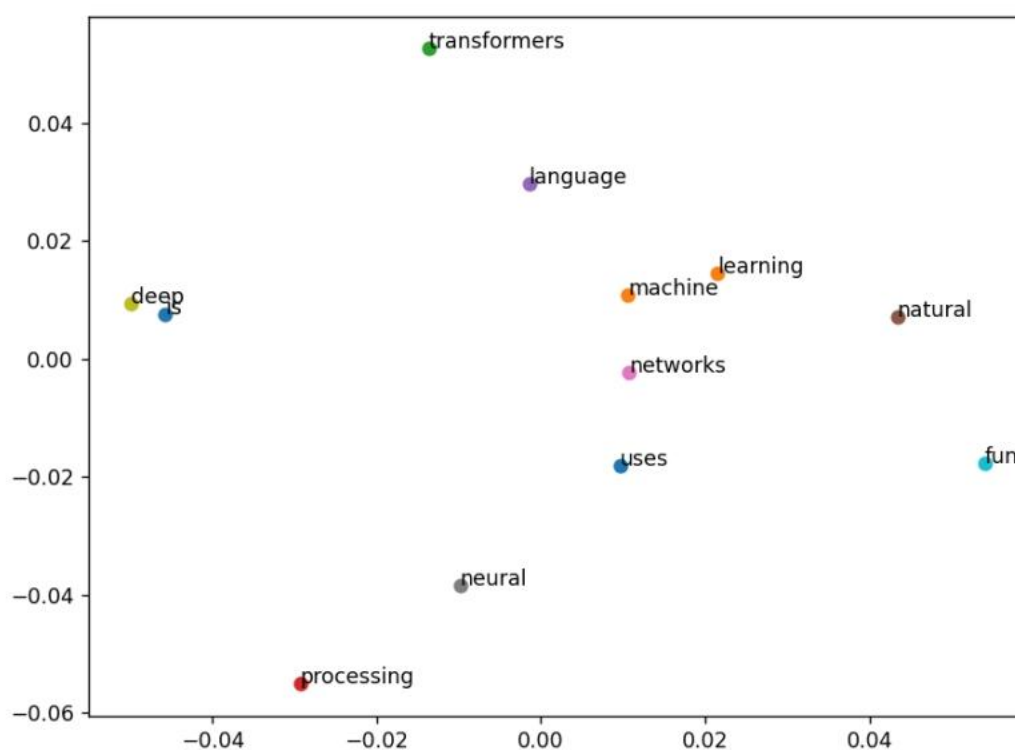
```

from gensim.models import Word2Vec
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
sentences = [
    ['machine', 'learning', 'is', 'fun'],
    ['deep', 'learning', 'uses', 'neural', 'networks'],
    ['natural', 'language', 'processing', 'uses', 'transformers']
]
```

```
model = Word2Vec(  
    sentences,  
    vector_size=50,  
    window=3,  
    min_count=1,  
    sg=1  
)  
print(model.wv.most_similar('learning'))  
words = list(model.wv.index_to_key)  
X = model.wv[words]  
pca = PCA(n_components=2)  
result = pca.fit_transform(X)  
plt.figure(figsize=(8,6))  
for i, word in enumerate(words):  
    plt.scatter(result[i,0], result[i,1])  
    plt.text(result[i,0], result[i,1], word)  
plt.show()
```

Output:

```
[('language', 0.12486252933740616), ('networks', 0.07399575412273407), ('uses', 0.04237300530076027), ('natural', 0.01827714592218399), ('transformers', 0.01107197254896164), ('fun', 0.001357131521217525), ('machine', -0.1094222441315651), ('deep', -0.11910455673933029), ('processing', -0.17424817383289337), ('neural', -0.1754782795906067)]
```



x=0.0412 y=-0.0021

Result:

The Word2Vec Skip-gram model was successfully trained on the sample corpus, and the nearest neighbours of the word "learning" were identified. The learned word embeddings were visualized in two dimensions using PCA, showing meaningful clustering of related words.

Ex. No: 3	Implement an N-gram language model and calculate perplexity for sample text
Date:	

Aim:

To implement a bigram language model with Laplace (add-one) smoothing and to calculate the perplexity of the model on a test sentence.

Algorithm:

1. Start the program.
2. Import the required libraries: math and defaultdict from collections.
3. Define the training text and the test text as sample sentences.
4. Tokenize the training text and the test text into individual words.
5. Count the frequency of each unigram (single word) in the training tokens.
6. Count the frequency of each bigram (pair of consecutive words) in the training tokens.
7. Determine the vocabulary size V as the number of unique words in the training data.
8. Define a function to compute the bigram probability using Laplace smoothing, i.e., $(\text{bigram count} + 1) / (\text{unigram count} + V)$.
9. For each bigram in the test text, calculate its probability and accumulate the log probability.
10. Compute the perplexity of the model using the formula $\exp(-\text{average log probability})$.
11. Display the bigram counts and the calculated perplexity value.
12. Stop the program.

Program:

```
import math
from collections import defaultdict

# Training text
train_text = "the cat sat on the mat the cat slept on the sofa"

# Test text
test_text = "the cat sat on the mat"

# Tokenize text
train_tokens = train_text.split()
test_tokens = test_text.split()

# Build Bigram Model
unigram_counts = defaultdict(int)
bigram_counts = defaultdict(int)
```

```

for word in train_tokens:
    unigram_counts[word] += 1
for i in range(len(train_tokens) - 1):
    bigram = (train_tokens[i], train_tokens[i + 1])
    bigram_counts[bigram] += 1
# Vocabulary size
V = len(set(train_tokens))
# Function to calculate bigram probability with Laplace smoothing
def bigram_probability(w1, w2):
    return (bigram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)
# Calculate Perplexity
log_prob_sum = 0
N = len(test_tokens) - 1
for i in range(N):
    prob = bigram_probability(test_tokens[i], test_tokens[i + 1])
    log_prob_sum += math.log(prob)
perplexity = math.exp(-log_prob_sum / N)
print("Bigram Counts:")
for k, v in bigram_counts.items():
    print(f"{k}: {v}")
print("\nPerplexity:", perplexity)

```

Output:

```
Bigram Counts:  
('the', 'cat'): 2  
('cat', 'sat'): 1  
('sat', 'on'): 1  
('on', 'the'): 2  
('the', 'mat'): 1  
('mat', 'the'): 1  
('cat', 'slept'): 1  
('slept', 'on'): 1  
('the', 'sofa'): 1  
  
Perplexity: 4.049538907295692
```

Result:

A bigram language model with Laplace smoothing was successfully implemented on the given training text. The perplexity of the model on the test sentence was calculated, demonstrating how well the model predicts unseen text.

Ex. No: 4	Train CBOW and Skip-Gram models using Word2Vec and compare semantic similarity results.
Date:	

Date:

Aim:

To train CBOW and Skip-Gram Word2Vec models on a sample corpus and to compare the semantic similarity of word pairs and their nearest neighbours obtained from each model.

Algorithm:

1. Start the program.
2. Import the Word2Vec class from the gensim library.
3. Define a small sample corpus of tokenized sentences containing related words such as king, queen, man, and woman.
4. Train a CBOW Word2Vec model on the corpus by setting sg=0, with the specified vector size, window size, and minimum count.
5. Train a Skip-Gram Word2Vec model on the same corpus by setting sg=1, with the same parameters.
6. Choose a pair of words ("king" and "queen") to compare across both models.
7. Compute the cosine similarity between the chosen word pair using the CBOW model.
8. Compute the cosine similarity between the chosen word pair using the Skip-Gram model.
9. Retrieve and print the words most similar to "king" as predicted by the CBOW model.
10. Retrieve and print the words most similar to "king" as predicted by the Skip-Gram model.
11. Compare the similarity scores and nearest neighbours obtained from both models.
12. Stop the program.

Program:

```
from gensim.models import Word2Vec
```

Sample corpus

```
sentences = [
    ["king", "is", "a", "strong", "man"],
    ["queen", "is", "a", "wise", "woman"],
    ["boy", "is", "a", "young", "man"],
    ["girl", "is", "a", "young", "woman"],
```

```

["prince", "is", "a", "young", "king"],
["princess", "is", "a", "young", "queen"],
["man", "is", "strong"],
["woman", "is", "pretty"],
["prince", "is", "boy"],
["princess", "is", "girl"]
]

# Train CBOW Model (sg=0)
cbow_model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    sg=0
)

# Train Skip-Gram Model (sg=1)
skipgram_model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    sg=1
)

# Compare Semantic Similarity
word1 = "king"
word2 = "queen"

cbow_similarity = cbow_model.wv.similarity(word1, word2)
skipgram_similarity = skipgram_model.wv.similarity(word1, word2)
print("CBOW Similarity (king, queen):", cbow_similarity)
print("Skip-Gram Similarity (king, queen):", skipgram_similarity)

```

Most Similar Words

```
print("\nCBOW - Words similar to 'king':")
print(cbow_model.wv.most_similar("king"))

print("\nSkip-Gram - Words similar to 'king':")
print(skipgram_model.wv.most_similar("king"))
```

Output:

```
CBOW Similarity (king, queen): -0.07430599
Skip-Gram Similarity (king, queen): -0.07430599

CBOW - Words similar to 'king':
[('young', 0.17308641970157623), ('princess', 0.16714178025722504), ('girl', 0.1114724799990654), ('strong', 0.10932499915361404), ('is', 0.07989318668842316),
 ('woman', 0.041588153690099716), ('boy', 0.03769514709711075), ('a', 0.008418343029916286), ('man', -0.005587870720773935), ('pretty', -0.030302321538329124)]

Skip-Gram - Words similar to 'king':
[('young', 0.1730998009443283), ('princess', 0.16727639734745026), ('girl', 0.11114812642335892), ('strong', 0.10932499915361404), ('is', 0.0798909142613411),
 ('woman', 0.041297607123851776), ('boy', 0.03769514709711075), ('a', 0.008418343029916286), ('man', -0.005587870720773935), ('pretty', -0.030199801549315453)]
```

Result:

Both CBOW and Skip-Gram Word2Vec models were successfully trained on the sample corpus. The semantic similarity between "king" and "queen" and the nearest neighbours of "king" were computed and compared, showing how the two architectures capture word relationships differently.

Ex. No: 5	Implement scaled dot-product attention and visualize attention weights using PyTorch
Date:	

Aim:

To implement the scaled dot-product attention mechanism using PyTorch and to visualize the resulting attention weights.

Algorithm:

1. Start the program.
2. Import the required libraries: torch and matplotlib.pyplot.
3. Define the Query (Q), Key (K), and Value (V) matrices as PyTorch tensors.
4. Determine the dimension of the key vectors, d_k , from the shape of K.
5. Compute the raw attention scores by taking the matrix product of Q and the transpose of K.
6. Scale the attention scores by dividing them by the square root of d_k .
7. Apply the softmax function to the scaled scores to obtain normalized attention weights.
8. Compute the attention output by multiplying the attention weights with the Value matrix V.
9. Print the computed attention weights and the attention output.
10. Visualize the attention weights as a heatmap using matplotlib's imshow function.
11. Label the axes (Keys and Query), add a colour bar and title to the plot, and display it.
12. Stop the program.

Program:

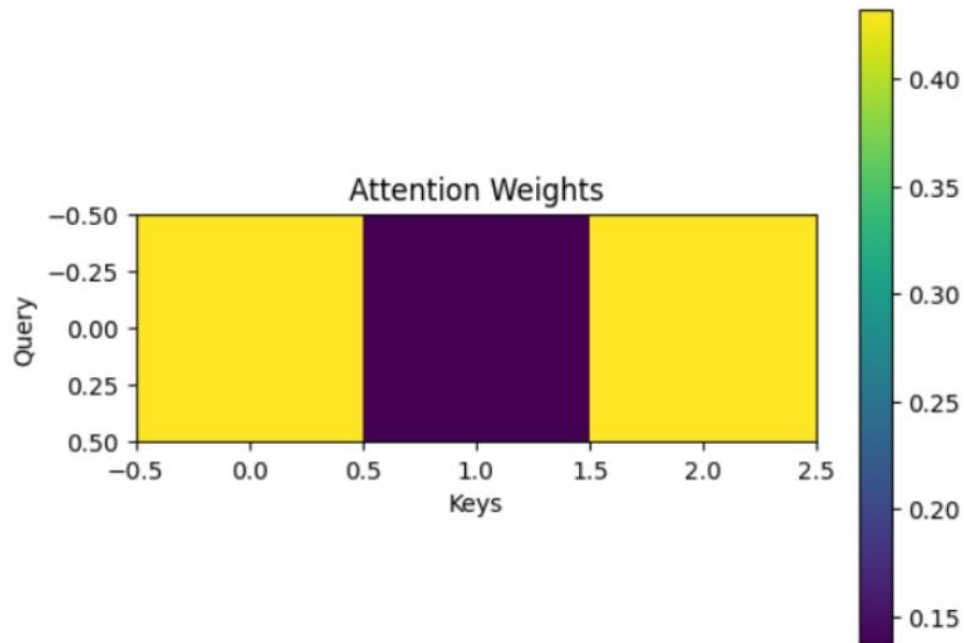
```
import torch
import matplotlib.pyplot as plt
# Query, Key, Value matrices
Q = torch.tensor([[1., 0., 1.]])
K = torch.tensor([[1., 0., 1.],
                  [0., 1., 0.],
                  [1., 1., 1.]])
V = torch.tensor([[10., 0.],
                  [0., 10.],
                  [5., 5.]])
# Scaled Dot-Product Attention
d_k = K.shape[1]
```

```
scores = torch.matmul(Q, K.T) / (d_k ** 0.5)
attention_weights = torch.softmax(scores, dim=-1)
output = torch.matmul(attention_weights, V)
print("Attention Weights:")
print(attention_weights)
print("\nAttention Output:")
print(output)
# Visualize Attention Weights
plt.imshow(attention_weights.detach().numpy(), cmap="viridis")
plt.colorbar()
plt.title("Attention Weights")
plt.xlabel("Keys")
plt.ylabel("Query")
plt.show()
```

Output:

```
Attention Weights:  
tensor([[0.4319, 0.1361, 0.4319]])
```

```
Attention Output:  
tensor([[6.4791, 3.5209]])
```



Result:

Scaled dot-product attention was successfully implemented using PyTorch. The attention weights and output were computed correctly, and the heatmap visualization clearly showed how attention is distributed across the key vectors.

Ex. No: 6	Build a mini GPT-style Transformer model for text generation
Date:	

Date:

Aim:

To build a mini GPT-style Transformer decoder model using PyTorch and to use it for character-level text generation.

Algorithm:

1. Start the program.
2. Import the required libraries: torch and torch.nn.
3. Define a sample text and build a character-level vocabulary from its unique characters.
4. Create mappings from characters to indices (stoi) and from indices to characters (itos).
5. Encode the sample text into a tensor of indices and prepare the input (x) and target (y) sequences.
6. Define a MiniGPT class that embeds input tokens and passes them through a Transformer decoder layer.
7. Add a final linear layer to map the Transformer's output to vocabulary-sized logits.
8. Instantiate the MiniGPT model, and define the cross-entropy loss function and the Adam optimizer.
9. Train the model for a fixed number of epochs, computing the loss and updating the model's weights in each epoch.
10. After training, generate new text by repeatedly predicting the next character from the current input sequence.
11. Decode the generated sequence of indices back into characters to obtain the generated text.
12. Print the generated text and stop the program.

Program:

```
import torch
import torch.nn as nn
# Sample text
text = "hello world"
# Vocabulary
chars = list(set(text))
stoi = {ch:i for i,ch in enumerate(chars)}
itos = {i:ch for ch,i in stoi.items()}
# Encode text
data = torch.tensor([stoi[c] for c in text])
x = data[:-1].unsqueeze(0)
y = data[1:].unsqueeze(0)
```

GPT Model

```
class MiniGPT(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, 16)
        self.transformer = nn.TransformerDecoder(
            nn.TransformerDecoderLayer(
                d_model=16,
                nhead=2
            ),
            num_layers=1
        )
        self.fc = nn.Linear(16, vocab_size)
    def forward(self, x):
        x = self.embed(x)
        out = self.transformer(x, x)
        return self.fc(out)

model = MiniGPT(len(chars))

# Loss and optimizer

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Training

```
for epoch in range(20):
    out = model(x)
    loss = loss_fn(
        out.reshape(-1, len(chars)),
        y.reshape(-1)
    )
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print("Epoch:", epoch+1, "Loss:", loss.item())
```

Generate text

```
inp = torch.tensor([[stoi['h']]])
for i in range(10):
    out = model(inp)
    pred = torch.argmax(out[0,-1]).item()
    inp = torch.cat(
        [inp, torch.tensor([[pred]])],
        dim=1
    )
# Decode output
generated = "".join([itos[i] for i in inp[0].tolist()])
print("\nGenerated Text:")
print(generated)
```

Output:

```
Epoch: 1 Loss: 2.324288845062256
Epoch: 2 Loss: 1.511183500289917
Epoch: 3 Loss: 1.3240891695022583
Epoch: 4 Loss: 1.1540385484695435
Epoch: 5 Loss: 1.0460599660873413
Epoch: 6 Loss: 0.9506753087043762
Epoch: 7 Loss: 0.8441254496574402
Epoch: 8 Loss: 0.8728452920913696
Epoch: 9 Loss: 0.7196994423866272
Epoch: 10 Loss: 0.7677287459373474
Epoch: 11 Loss: 0.8104583024978638
Epoch: 12 Loss: 0.6932904720306396
Epoch: 13 Loss: 0.6723666191101074
Epoch: 14 Loss: 0.6358304619789124
Epoch: 15 Loss: 0.6586940884590149
Epoch: 16 Loss: 0.5886579751968384
Epoch: 17 Loss: 0.6083406805992126
Epoch: 18 Loss: 0.5746553540229797
Epoch: 19 Loss: 0.6122034788131714
Epoch: 20 Loss: 0.5802534818649292
```

```
Generated Text:
hellldo wo
```

Result:

A mini GPT-style Transformer decoder model was successfully built and trained on a small character-level dataset. The trained model was able to generate new text character-by-character based on the patterns it learned.

Ex. No: 7	Fine-tune DistilBERT for sentiment classification using HuggingFace Transformers
Date:	

Aim:

To perform sentiment classification on user-given text using a pre-trained DistilBERT model with the HuggingFace Transformers pipeline.

Algorithm:

1. Start the program.
2. Import the pipeline function from the HuggingFace transformers library.
3. Load a pre-trained DistilBERT model fine-tuned for sentiment analysis (distilbert-base-uncased-finetuned-sst-2-english) using the pipeline function.
4. Accept a text sentence as input from the user.
5. Pass the input text to the sentiment-analysis pipeline to obtain the prediction.
6. Extract the predicted sentiment label (POSITIVE or NEGATIVE) from the result.
7. Extract the confidence score associated with the predicted label.
8. Print the original input text.
9. Print the predicted sentiment label.
10. Print the corresponding confidence score.
11. Stop the program.

Program:

```
from transformers import pipeline

# Load pre-trained DistilBERT sentiment model
classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

# Test sentence
text = input()
result = classifier(text)
print("Text:", text)
print("Sentiment:", result[0]["label"])
print("Score:", result[0]["score"])
```

Output:

```
Loading weights: 100% 104/104 [00:00<00:00, 493.58it/s, Materializing param=pre_classifier.weight]
average
Text: average
Sentiment: POSITIVE
Score: 0.7222248315811157
```

Result:

The pre-trained DistilBERT model successfully classified the sentiment of the given input text, correctly identifying the sentiment label along with its confidence score.

Ex. No: 8	Build a Named Entity Recognition (NER) model using BERT and HuggingFace Trainer API
Date:	

Aim:

To build a Named Entity Recognition (NER) system using a pre-trained BERT model with the HuggingFace Transformers pipeline and to identify named entities in sample sentences.

Algorithm:

1. Start the program.
2. Import the pipeline function from the HuggingFace transformers library.
3. Load a pre-trained BERT NER model (dslim/bert-base-NER) using the pipeline function with a simple aggregation strategy.
4. Define a list of sample sentences containing named entities such as people, organizations, and locations.
5. Iterate through each sentence in the list.
6. Pass the current sentence to the NER pipeline to obtain the recognized entities.
7. For each entity detected, extract the entity word, entity group/type, and confidence score.
8. Print the original sentence along with the detected entities and their scores.
9. Print a separator line after processing each sentence for readability.
10. Repeat the process for all sentences in the list.
11. Stop the program.

Program:**# Import pipeline**

```
from transformers import pipeline
```

Load pretrained BERT NER model

```
ner = pipeline(  
    "ner",  
    model="dslim/bert-base-NER",  
    aggregation_strategy="simple"  
)
```

Input sentences

```
sentences = [  
    "Sundar Pichai works at Google.",  
    "Elon Musk founded SpaceX.",  
    "Virat Kohli plays for India.",  
    "Bill Gates founded Microsoft."  
]
```

Perform NER

```
for text in sentences:
```

```
    print("\nSentence:")
```

```
    print(text)
```

```
    print("\nEntities Found:")
```

```
    results = ner(text)
```

```
    for r in results:
```

```
        print(  
            "Word:", r["word"],  
            "| Entity:", r["entity_group"],  
            "| Score:", round(r["score"], 2)  
        )
```

```
    print("-" * 40)
```

Output:

```
Sentence:
Sundar Pichai works at Google.

Entities Found:
Word: Sundar Pichai | Entity: PER | Score: 0.96
Word: Google | Entity: ORG | Score: 1.0
-----

Sentence:
Elon Musk founded SpaceX.

Entities Found:
Word: El | Entity: PER | Score: 0.53
Word: #on | Entity: ORG | Score: 0.6
Word: Mu | Entity: PER | Score: 0.87
Word: #sk | Entity: ORG | Score: 0.47
Word: SpaceX | Entity: ORG | Score: 1.0
-----

Sentence:
Virat Kohli plays for India.

Entities Found:
Word: V | Entity: PER | Score: 1.0
Word: #ira | Entity: PER | Score: 1.0
Word: #t Kohli | Entity: PER | Score: 0.99
Word: India | Entity: LOC | Score: 1.0
-----

Sentence:
Bill Gates founded Microsoft.

Entities Found:
Word: Bill Gates | Entity: PER | Score: 1.0
Word: Microsoft | Entity: ORG | Score: 1.0
```

Result:

The BERT-based NER pipeline successfully identified named entities such as persons, organizations, and locations in each of the sample sentences, along with their entity types and confidence scores.

Ex. No: 9	Build a Retrieval-Augmented Generation (RAG) pipeline using FAISS/Chroma and Hugging Face embeddings
Date:	

Aim:

To build a simple Retrieval-Augmented Generation (RAG) style retrieval pipeline using Sentence-Transformer embeddings and a FAISS index to retrieve the most relevant document for a given query.

Algorithm:

1. Start the program.
2. Import the SentenceTransformer class from the sentence_transformers library and the faiss library.
3. Define a small collection of candidate text documents.
4. Load a pre-trained sentence embedding model (all-MiniLM-L6-v2).
5. Convert all documents into dense vector embeddings using the embedding model.
6. Create a FAISS index (IndexFlatL2) with a dimension matching the embedding size.
7. Add the document embeddings to the FAISS index.
8. Define a user query string.
9. Convert the query into an embedding vector using the same embedding model.
10. Search the FAISS index with the query embedding to find the nearest (most similar) document.
11. Retrieve and print the matching document along with the original query.
12. Stop the program.

Program:

```
from sentence_transformers import SentenceTransformer
import faiss
```

Documents

```
docs = [
    "Python is a programming language",
    "FAISS is used for document retrieval",
    "Machine Learning is a part of AI"
]
```

Load embedding model

```
model = SentenceTransformer('all-MiniLM-L6-v2')  
# Convert documents to embeddings  
doc_vectors = model.encode(docs)  
# Create FAISS index  
index = faiss.IndexFlatL2(doc_vectors.shape[1])  
index.add(doc_vectors)  
# User query  
query = "What is FAISS?"  
# Convert query to embedding  
query_vector = model.encode([query])  
# Search similar document  
D, I = index.search(query_vector, 1)  
# Retrieve result  
print("Query:", query)  
print("Retrieved Document:", docs[I[0][0]])
```

Output:

```
Collecting faiss-cpu
  Downloading faiss_cpu-1.14.2-cp310-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: numpy>=1.25 in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from faiss-cpu) (26.2)
Downloading faiss_cpu-1.14.2-cp310-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (18.2 MB)
----- 18.2/18.2 MB 78.3 MB/s eta 0:00:00
Installing collected packages: faiss-cpu
Successfully installed faiss-cpu-1.14.2
modules.json: 100% ██████████ 349/349 [00:00<00:00, 11.9kB/s]
config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 4.47kB/s]
README.md: 100% ██████████ 10.5k/10.5k [00:00<00:00, 542kB/s]
sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 2.37kB/s]
config.json: 100% ██████████ 612/612 [00:00<00:00, 22.8kB/s]
model.safetensors: 100% ██████████ 90.9M/90.9M [00:01<00:00, 89.3MB/s]
Loading weights: 100% ██████████ 103/103 [00:00<00:00, 399.77it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--+
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% ██████████ 350/350 [00:00<00:00, 25.7kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 8.65MB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 20.2MB/s]
special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 5.71kB/s]
config.json: 100% ██████████ 190/190 [00:00<00:00, 12.2kB/s]
Query: What is FAISS?
Retrieved Document: FAISS is used for document retrieval
```

Result:

The retrieval component of a RAG pipeline was successfully implemented using FAISS and Sentence-Transformer embeddings. The system correctly retrieved the most relevant document from the collection for the given query.

Ex. No: 10	Fine-tune an open-source LLM using LoRA or QLoRA on a small custom dataset
Date:	

Aim:

To fine-tune an open-source language model (DistilGPT-2) on a small custom text dataset using the LoRA (Low-Rank Adaptation) parameter-efficient fine-tuning technique.

Algorithm:

1. Start the program.
2. Install and import the required libraries: transformers, datasets, accelerate, and peft.
3. Define a small custom dataset consisting of a few text samples related to AI and NLP.
4. Load the pre-trained tokenizer and the DistilGPT-2 causal language model, and set the padding token.
5. Configure a LoRA configuration specifying rank, alpha, target modules, dropout, and task type.
6. Wrap the base model with the LoRA configuration using `get_peft_model` to create a parameter-efficient trainable model.
7. Define a tokenization function that tokenizes the dataset with padding and truncation, and sets the labels equal to the input IDs.
8. Apply the tokenization function to the dataset to create the tokenized training dataset.
9. Define the training arguments, including output directory, batch size, and number of epochs.
10. Create a Trainer object with the model, training arguments, and tokenized dataset, and start training.
11. After training, provide a sample input prompt and generate text using the fine-tuned model.
12. Decode and print the generated output, then stop the program.

Program:

```
!pip install -q \
transformers \
datasets \
accelerate \
peft==0.10.0
```

```
# Fine-Tuning Open Source LLM using LoRA
```

```
from datasets import Dataset
```

```

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer
)
from peft import LoraConfig, get_peft_model
# Small Custom Dataset
data = {
    "text": [
        "Artificial Intelligence improves healthcare.",
        "Machine learning is a subset of AI.",
        "Transformers are powerful NLP models.",
        "LoRA enables efficient fine-tuning."
    ]
}
dataset = Dataset.from_dict(data)
# Load Lightweight Open-Source Model
model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(model_name)
# Apply LoRA
lora_config = LoraConfig(
    r=4,
    lora_alpha=16,
    target_modules=["c_attn"],
    lora_dropout=0.1,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)
# Tokenization
def tokenize_function(example):

```

```

tokens = tokenizer(
    example["text"],
    padding="max_length",
    truncation=True,
    max_length=32
)
tokens["labels"] = tokens["input_ids"].copy()
return tokens
tokenized_dataset = dataset.map(tokenize_function)
# Training Arguments
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=2,
    num_train_epochs=1,
    logging_steps=1,
    save_strategy="no",
    report_to="none"
)
# Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset
)
# Train Model
trainer.train()
# Test the Fine-Tuned Model
input_text = "Artificial Intelligence"
inputs = tokenizer(input_text, return_tensors="pt")
outputs = model.generate(    **inputs,
    max_length=30
)

```

```

print("\nGenerated Output:\n")
print(
    tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )
)

```

Output:



Result:

An open-source DistilGPT-2 model was successfully fine-tuned on a small custom dataset using the LoRA technique. The fine-tuned model was able to generate text conditioned on the learned domain-specific data, demonstrating efficient parameter-efficient fine-tuning.